

STRIPE BILLING SERVICE

Complete Testing Document

Version 1.0

Date: March 2, 2026

Technology Stack: ASP.NET Core 9 | React 19 | SQL Server | Stripe API

Classification: Confidential

Table of Contents

Table of Contents.....	2
1. Introduction	4
1.1 Purpose	4
1.2 Application Overview	4
1.3 Scope of Testing.....	4
1.4 Test Environment	4
1.5 Roles and Permissions Matrix	5
2. Authentication and Authorization Testing	6
2.1 JWT Authentication	6
2.2 API Key Authentication	6
2.3 HMAC Webhook Signature Verification	7
2.4 Role-Based Access Control	7
3. Multi-Tenant Isolation Testing	8
4. Payment Processing Testing.....	9
4.1 Checkout Sessions.....	9
4.2 Payment Intents.....	9
4.3 Idempotency.....	9
5. Subscription Management Testing	11
5.1 Subscription Plans.....	11
5.2 Customer Subscriptions.....	11
6. Customer Management Testing	12
7. Invoice and Billing Testing.....	13
7.1 Invoices	13
7.2 Refunds	13
7.3 Coupons and Promotion Codes.....	13
8. Webhook System Testing	15
8.1 Inbound Webhooks (from Stripe).....	15
8.2 Outbound Webhooks (to Clients).....	15
9. Usage Billing, Credits, and Dunning Testing	17
9.1 Usage-Based Billing	17
9.2 Customer Credits.....	17
9.3 Dunning (Payment Recovery).....	17
10. Middleware Testing	19
10.1 Rate Limiting	19

10.2 Tenant Resolution Middleware	19
10.3 Idempotency Middleware	19
10.4 Request Logging Middleware	19
11. Background Services Testing	21
12. Analytics and Dashboard Testing	22
12.1 Revenue Analytics	22
12.2 SignalR Real-Time Updates	22
12.3 Export	22
13. Stripe Connect and Tax Configuration Testing	24
13.1 Stripe Connect	24
13.2 Tax Configuration	24
14. Email Template and Notification Testing	25
15. SuperAdmin Module Testing	26
16. Frontend UI Testing	27
16.1 Authentication UI	27
16.2 Dashboard	27
16.3 Billing Pages	27
16.4 Gateway and Admin Pages	28
16.5 SuperAdmin Pages	29
17. Input Validation Testing	30
18. Security Testing	31
19. Performance and Load Testing	32
20. Integration and End-to-End Testing	33
20.1 Critical End-to-End Flows	33
20.2 Stripe Integration Tests	33
21. Docker and Deployment Testing	35
22. Existing Unit Tests and Coverage Gaps	36
22.1 Current Test Coverage	36
22.2 Critical Coverage Gaps	36
22.3 Recommended Test Coverage Targets	36
23. Test Execution Plan	38
23.1 Execution Phases	38
23.2 Tools and Frameworks	38
23.3 Test Data Strategy	38
23.4 Defect Severity Classification	38

1. Introduction

1.1 Purpose

This document provides a complete testing strategy and test case catalog for the Stripe Billing Service application. It covers all functional modules, integration points, security layers, and non-functional requirements to ensure the system operates reliably in a production multi-tenant environment.

1.2 Application Overview

The Stripe Billing Service is a multi-tenant SaaS billing platform built on ASP.NET Core 9 (backend) and React 19 (frontend). It integrates with Stripe APIs for payment processing, subscription management, invoicing, and revenue analytics. The system supports four user roles (SuperAdmin, Admin, Manager, Viewer) with complete tenant data isolation.

1.3 Scope of Testing

The testing scope encompasses the following areas:

- Backend API endpoints (29 controllers, 30+ services)
- Frontend UI pages (20+ pages across billing, analytics, gateway, and admin)
- Stripe API integration (Checkout, Payment Intents, Subscriptions, Webhooks)
- Authentication and authorization (JWT, API Keys, HMAC, RBAC)
- Middleware pipeline (Rate Limiting, Idempotency, Tenant Resolution, Request Logging)
- Background services (Webhook Dispatch, Retry, Dunning Processor)
- Real-time communication (SignalR Dashboard Hub)
- Data layer (Entity Framework Core, SQL Server, multi-tenant queries)
- Security testing (injection, XSS, CSRF, data isolation)
- Performance, load, and scalability testing

1.4 Test Environment

TC ID	Test Case	Steps / Input	Expected Result	Priority
ENV-01	Development Local	Docker Compose: SQL Server + API + Frontend + Seq	All services running locally	Critical
ENV-02	Stripe Test Mode	Use pk_test_ and sk_test_ keys with Stripe CLI for webhooks	Test transactions without real charges	Critical
ENV-03	Database	SQL Server 2022 with EF Core migrations, seeded demo data	2 tenants, 6 users, 15 customers, 75+ payments	High
ENV-04	Test Framework	xUnit + Moq + FluentAssertions + EF InMemory + WebApplicationFactory	Unit, integration, and E2E test support	High

1.5 Roles and Permissions Matrix

All testing must validate proper role enforcement across these four roles:

Feature Area	SuperAdmin	Admin	Manager	Viewer	Notes
Dashboard	Full	Full	Full	Read	Viewer sees analytics only
Payments	Full	Full	Create/Read	Read	
Subscriptions	Full	Full	Create/Read	Read	
Customers	Full	Full	Create/Read	Read	
Users / Settings	Full	Full	Denied	Denied	Admin+ only
Tenant Management	Full	Denied	Denied	Denied	SuperAdmin only
Invoices / Refunds	Full	Full	Create/Read	Read	
API Keys / Webhooks	Full	Full	Read	Read	

2. Authentication and Authorization Testing

2.1 JWT Authentication

TC ID	Test Case	Steps / Input	Expected Result	Priority
AUTH-01	Login with valid credentials	POST /api/v1/auth/login with superadmin@techflow.com / Demo@123	200 OK, JWT token returned with user data and expiresIn	Critical
AUTH-02	Login with invalid password	POST /api/v1/auth/login with valid email, wrong password	401 Unauthorized response	Critical
AUTH-03	Login with non-existent email	POST /api/v1/auth/login with unknown@email.com	401 Unauthorized, no user data leak	Critical
AUTH-04	Access protected endpoint without token	GET /api/v1/customers without Authorization header	401 Unauthorized	Critical
AUTH-05	Access with expired JWT token	Use a token that has expired (modify exp claim)	401 Unauthorized	Critical
AUTH-06	Access with tampered JWT token	Modify payload of valid JWT, keep signature	401 Unauthorized (signature invalid)	Critical
AUTH-07	Token refresh with valid refresh token	POST /api/v1/auth/refresh-token with valid token	200 OK, new JWT token issued	High
AUTH-08	Token refresh with expired refresh token	POST /api/v1/auth/refresh-token with expired token	401 Unauthorized	High
AUTH-09	Get current user profile	GET /api/v1/auth/me with valid token	200 OK, returns user details matching token claims	High
AUTH-10	Change password	POST /api/v1/auth/change-password with current and new password	200 OK, old password no longer works	High
AUTH-11	Register new user	POST /api/v1/auth/register with complete valid payload	201 Created, new user accessible	High
AUTH-12	Register with duplicate email	POST /api/v1/auth/register with already-registered email	400 Bad Request, duplicate error	Medium

2.2 API Key Authentication

TC ID	Test Case	Steps / Input	Expected Result	Priority
APIKEY-01	Access with valid API key	Send request with X-API-Key and X-Tenant-Id headers	200 OK, request processed	Critical
APIKEY-02	Access with invalid API key	Send request with incorrect X-API-Key value	401 Unauthorized	Critical
APIKEY-03	Access with revoked API key	Revoke key via admin, then use it	401 Unauthorized	High
APIKEY-04	API key without tenant header	Send X-API-Key without X-Tenant-Id	400 Bad Request	High

APIKEY-05	Create new API key	POST /api/v1/api-keys with name and permissions	201 Created, key value returned once	High
APIKEY-06	List API keys (key values hidden)	GET /api/v1/api-keys as Admin	200 OK, keys listed with masked values	Medium

2.3 HMAC Webhook Signature Verification

TC ID	Test Case	Steps / Input	Expected Result	Priority
HMAC-01	Valid HMAC signature on webhook	Compute HMAC-SHA256 of payload with secret, send in X-Signature	Webhook accepted and processed	Critical
HMAC-02	Invalid HMAC signature	Send webhook with wrong X-Signature value	401 Unauthorized, webhook rejected	Critical
HMAC-03	Missing signature header	Send webhook payload without X-Signature header	401 Unauthorized	High
HMAC-04	Replay attack (old timestamp)	Send webhook with valid signature but old X-Timestamp	Rejected if timestamp tolerance exceeded	High

2.4 Role-Based Access Control

TC ID	Test Case	Steps / Input	Expected Result	Priority
RBAC-01	Viewer cannot create payment	Login as Viewer, POST /api/v1/payments/checkout	403 Forbidden	Critical
RBAC-02	Manager cannot manage users	Login as Manager, POST /api/v1/users	403 Forbidden	Critical
RBAC-03	Admin cannot access SuperAdmin endpoints	Login as Admin, GET /api/v1/super-admin/tenants	403 Forbidden	Critical
RBAC-04	SuperAdmin can access all endpoints	Login as SuperAdmin, access all controller routes	All return 200/201 (not 403)	Critical
RBAC-05	Viewer can read payments	Login as Viewer, GET /api/v1/payments	200 OK, list of payments returned	High
RBAC-06	Admin can manage settings	Login as Admin, PUT /api/v1/settings	200 OK, settings updated	High

3. Multi-Tenant Isolation Testing

Multi-tenant data isolation is a critical security requirement. Every data query must be scoped to the authenticated tenant. These tests ensure complete separation between TechFlow Solutions (Tenant 1) and Sunrise Dental Group (Tenant 2).

TC ID	Test Case	Steps / Input	Expected Result	Priority
MT-01	Tenant 1 cannot see Tenant 2 customers	Login as TechFlow Admin, GET /api/v1/customers	Only TechFlow customers returned (not Sunrise Dental)	Critical
MT-02	Tenant 2 cannot see Tenant 1 payments	Login as Sunrise Dental Admin, GET /api/v1/payments	Only Sunrise Dental payments returned	Critical
MT-03	Cross-tenant resource access by ID	Login as Tenant 1, GET /api/v1/customers/{tenant2_customer_id}	404 Not Found (not 403, to avoid info leak)	Critical
MT-04	Tenant header manipulation	Login as Tenant 1 user, send X-Tenant-Id of Tenant 2	Rejected or scoped to authenticated tenant only	Critical
MT-05	Subscription plan isolation	Login as each tenant, GET /api/v1/plans	Each tenant sees only their own plans	High
MT-06	Invoice isolation	Login as Tenant 1, GET /api/v1/invoices	Only Tenant 1 invoices returned	High
MT-07	Webhook subscription isolation	Login as each tenant, GET /api/v1/webhooks	Each tenant sees only their webhook configs	High
MT-08	Analytics data isolation	Login as each tenant, GET /api/v1/analytics/revenue	Revenue metrics reflect only that tenant's data	High
MT-09	Audit log isolation	Login as each tenant Admin, GET /api/v1/audit-logs	Only that tenant's audit entries visible	High
MT-10	Settings isolation	Update settings for Tenant 1, verify Tenant 2 unchanged	Settings are completely independent per tenant	Medium

4. Payment Processing Testing

4.1 Checkout Sessions

TC ID	Test Case	Steps / Input	Expected Result	Priority
PAY-01	Create checkout session	POST /api/v1/payments/checkout with valid customerId, planId, quantity	201 Created, Stripe checkout session URL returned	Critical
PAY-02	Checkout with invalid customer	POST /api/v1/payments/checkout with non-existent customerId	404 Not Found	High
PAY-03	Checkout with missing required fields	POST /api/v1/payments/checkout without planId	400 Bad Request with validation errors	High
PAY-04	Checkout session completion webhook	Complete checkout in Stripe test, receive checkout.session.completed webhook	Payment record created, SignalR notification sent	Critical

4.2 Payment Intents

TC ID	Test Case	Steps / Input	Expected Result	Priority
PAY-05	Create payment intent	POST /api/v1/payments/create-intent with amount, currency, customerId	201 Created, client_secret returned for frontend confirmation	Critical
PAY-06	Payment intent succeeded webhook	Trigger payment_intent.succeeded via Stripe CLI	Payment record updated to succeeded status	Critical
PAY-07	Payment intent failed webhook	Trigger payment_intent.payment_failed via Stripe test card 4000000000000002	Payment record updated to failed, dunning process may trigger	Critical
PAY-08	Get single payment by ID	GET /api/v1/payments/{id} with valid payment ID	200 OK, complete payment details returned	High
PAY-09	List payments with filters	GET /api/v1/payments?status=succeeded&period=30d	200 OK, filtered list with pagination	High
PAY-10	Payment analytics	GET /api/v1/payments/analytics?period=30d	200 OK, aggregated payment metrics	Medium

4.3 Idempotency

TC ID	Test Case	Steps / Input	Expected Result	Priority
PAY-11	Duplicate payment with idempotency key	POST /api/v1/payments/create-intent twice with same X-Idempotency-Key	Second request returns cached response, no duplicate charge	Critical
PAY-12	Different payload same idempotency key	POST with same key but different amount	409 Conflict or return original cached response	High
PAY-13	Idempotency key expiry	Use same key after TTL expiry period	New request processed normally	Medium

5. Subscription Management Testing

5.1 Subscription Plans

TC ID	Test Case	Steps / Input	Expected Result	Priority
SUB-01	Create subscription plan	POST /api/v1/plans with name, amount, interval, currency	201 Created, plan synced to Stripe	Critical
SUB-02	List all plans	GET /api/v1/plans	200 OK, all tenant plans (Starter \$9, Pro \$49, Business \$149, Enterprise \$499, etc.)	High
SUB-03	Update plan metadata	PUT /api/v1/plans/{id} with updated description	200 OK, plan updated locally and on Stripe	Medium
SUB-04	Create plan with invalid interval	POST /api/v1/plans with interval='weekly' (unsupported)	400 Bad Request, validation error	Medium

5.2 Customer Subscriptions

TC ID	Test Case	Steps / Input	Expected Result	Priority
SUB-05	Subscribe customer to plan	POST /api/v1/subscriptions with customerId, planId	201 Created, Stripe subscription active, invoice generated	Critical
SUB-06	Cancel subscription immediately	POST /api/v1/subscriptions/{id}/cancel with immediate=true	Subscription status changes to canceled	Critical
SUB-07	Cancel at period end	POST /api/v1/subscriptions/{id}/cancel with immediate=false	cancel_at_period_end set to true, remains active until period end	High
SUB-08	Update subscription (change plan)	PUT /api/v1/subscriptions/{id} with new planId	Proration calculated, subscription updated	High
SUB-09	List subscriptions with status filter	GET /api/v1/subscriptions?status=active	200 OK, only active subscriptions returned	High
SUB-10	Subscription lifecycle webhooks	Monitor customer.subscription.created, updated, deleted events	Local records updated for each event	Critical
SUB-11	Duplicate subscription prevention	Subscribe same customer to same plan twice	400 Bad Request or handled gracefully	High
SUB-12	Subscription with trial period	Subscribe customer to plan with trial_period_days	Status is 'trialing', no immediate charge	Medium
SUB-13	Past due subscription handling	Trigger payment failure on renewal	Status changes to past_due, dunning process starts	High

6. Customer Management Testing

TC ID	Test Case	Steps / Input	Expected Result	Priority
CUST-01	Create customer	POST /api/v1/customers with name, email, metadata	201 Created, synced to Stripe, customer record stored	Critical
CUST-02	Create customer with duplicate email	POST /api/v1/customers with existing email in same tenant	400 Bad Request, duplicate detected	High
CUST-03	Update customer details	PUT /api/v1/customers/{id} with updated name/email	200 OK, updated locally and on Stripe	High
CUST-04	List customers with search	GET /api/v1/customers?search=Global&page=1&pageSize=10	200 OK, filtered paginated list	High
CUST-05	Get customer with subscriptions	GET /api/v1/customers/{id}	200 OK, includes active subscriptions and payment history	Medium
CUST-06	Customer validation - invalid email	POST /api/v1/customers with invalid email format	400 Bad Request, email validation error	Medium
CUST-07	Customer portal session	POST /api/v1/portal/session with customerId	URL returned for Stripe Customer Portal	High

7. Invoice and Billing Testing

7.1 Invoices

TC ID	Test Case	Steps / Input	Expected Result	Priority
INV-01	Auto-invoice on subscription payment	Create subscription, verify invoice created	Invoice auto-generated with correct amount and line items	Critical
INV-02	List invoices with filters	GET /api/v1/invoices?status=paid&customerId={id}	200 OK, filtered invoice list	High
INV-03	Invoice webhook (invoice.paid)	Trigger invoice.paid event from Stripe	Local invoice status updated to paid	Critical
INV-04	Invoice with tax calculation	Create invoice for customer with tax config (EU reverse-charge)	Tax amount calculated correctly, tax exemptions applied	High
INV-05	Invoice items management	POST /api/v1/invoice-items with description, amount, invoiceId	201 Created, line item added to invoice	Medium

7.2 Refunds

TC ID	Test Case	Steps / Input	Expected Result	Priority
REF-01	Full refund	POST /api/v1/refunds with paymentId, full amount	201 Created, refund processed on Stripe, payment status updated	Critical
REF-02	Partial refund	POST /api/v1/refunds with paymentId, partial amount	201 Created, partial refund processed	High
REF-03	Refund exceeding payment amount	POST /api/v1/refunds with amount > original payment	400 Bad Request, amount exceeds available	High
REF-04	Duplicate refund prevention	Issue same full refund twice	Second request rejected or handled idempotently	High
REF-05	Refund webhook (charge.refunded)	Trigger charge.refunded from Stripe	Local refund record updated	High
REF-06	List refunds with filters	GET /api/v1/refunds?status=succeeded	200 OK, filtered refund list	Medium

7.3 Coupons and Promotion Codes

TC ID	Test Case	Steps / Input	Expected Result	Priority
-------	-----------	---------------	-----------------	----------

CPN-01	Create percentage coupon	POST /api/v1/coupons with percent_off=20, duration=once	201 Created, coupon available on Stripe	High
CPN-02	Create fixed-amount coupon	POST /api/v1/coupons with amount_off=500, currency=usd	201 Created	High
CPN-03	Apply coupon to subscription	Apply WELCOME20 code to new subscription	20% discount applied on first invoice	High
CPN-04	Apply expired coupon	Apply coupon past its expiration date	400 Bad Request, coupon expired	Medium
CPN-05	Promotion code validation	Use promotion codes WELCOME20, ANNUAL50, PARTNER15, BFRIDAY30	Each code applies correct discount	Medium

8. Webhook System Testing

8.1 Inbound Webhooks (from Stripe)

TC ID	Test Case	Steps / Input	Expected Result	Priority
WH-01	Valid Stripe webhook signature	POST /api/v1/webhooks/stripe with valid Stripe-Signature header	200 OK, event processed	Critical
WH-02	Invalid Stripe webhook signature	POST /api/v1/webhooks/stripe with tampered signature	400 Bad Request, signature mismatch	Critical
WH-03	checkout.session.completed	Trigger via Stripe CLI: stripe trigger checkout.session.completed	Payment record created, outbound webhook dispatched	Critical
WH-04	payment_intent.succeeded	Trigger via Stripe CLI	Payment updated, SignalR notification sent	Critical
WH-05	customer.subscription.created	Create subscription in Stripe dashboard	Local subscription record created	High
WH-06	customer.subscription.updated	Update subscription in Stripe	Local record updated accordingly	High
WH-07	customer.subscription.deleted	Cancel subscription from Stripe	Local record marked as canceled	High
WH-08	invoice.paid	Trigger via Stripe CLI	Invoice marked paid, receipt flow triggered	High
WH-09	invoice.payment_failed	Trigger via Stripe test card for failure	Invoice marked failed, dunning initiated	High
WH-10	charge.refunded	Issue refund from Stripe dashboard	Refund record created locally	High
WH-11	charge.dispute.created	Simulate dispute event	Dispute record created, admin notified	Medium
WH-12	Duplicate webhook event	Send same event ID twice	Processed idempotently (no duplicate records)	High

8.2 Outbound Webhooks (to Clients)

TC ID	Test Case	Steps / Input	Expected Result	Priority
WH-13	Outbound webhook dispatch	Trigger payment event, verify dispatch to subscribed client URL	Webhook delivered with HMAC- SHA256 signature	Critical

WH-14	Webhook retry on failure (5 attempts)	Set client URL to return 500, verify retry behavior	Retried with exponential backoff up to 5 times	Critical
WH-15	Webhook delivery tracking	GET /api/v1/webhook-events	Delivery status, response codes, timestamps for each attempt	High
WH-16	Webhook subscription CRUD	Create, read, update, delete webhook subscriptions	Full CRUD operations working, URL validation enforced	High
WH-17	Webhook signature verification by client	Client verifies X-Signature header using shared secret	HMAC matches when computed correctly	High

9. Usage Billing, Credits, and Dunning Testing

9.1 Usage-Based Billing

TC ID	Test Case	Steps / Input	Expected Result	Priority
UB-01	Create usage record	POST /api/v1/usage-billing/records with subscriptionItemId, quantity, timestamp	201 Created, usage tracked for metered billing	High
UB-02	List usage records	GET /api/v1/usage-billing/records?subscriptionId={id}	200 OK, usage records with aggregated totals	High
UB-03	Usage billing at period end	Wait for billing cycle end (or simulate)	Invoice generated with metered usage charges	High
UB-04	Meter events (30 days API usage)	Verify seeded meter events display correctly	Usage data visible in dashboard with daily breakdown	Medium

9.2 Customer Credits

TC ID	Test Case	Steps / Input	Expected Result	Priority
CR-01	Add credit to customer	POST /api/v1/credits/{customerId}/add with amount=2500, currency=usd	Balance updated: previous + 2500	High
CR-02	Positive balance adjustment (credit)	POST /api/v1/credits/{customerId}/adjust with amount=1500	Type=credit, balance increases by 1500	High
CR-03	Negative balance adjustment (debit)	POST /api/v1/credits/{customerId}/adjust with amount=-3000	Type=debit, balance decreases by 3000	High
CR-04	Credit sets correct tenant and user	Add credit, verify tenantId and createdBy match authenticated context	TenantId and CreatedBy correctly populated	High
CR-05	Get credit balance	GET /api/v1/credits/{customerId}/balance	200 OK, current balance returned	Medium
CR-06	List credit transactions	GET /api/v1/credits/{customerId}/transactions	200 OK, chronological list of credits and debits	Medium

9.3 Dunning (Payment Recovery)

TC ID	Test Case	Steps / Input	Expected Result	Priority
DUN-01	Dunning schedule configuration (4 steps)	GET /api/v1/dunning/schedule	Returns 4 configured dunning steps	High

DUN-02	Automatic retry on failed payment	Trigger payment failure, wait for dunning processor background service	Retry email sent, payment retried per schedule	Critical
DUN-03	Dunning escalation	Allow all retries to fail sequentially	Each step sends appropriate notification, final step cancels subscription	High
DUN-04	Dunning processor background service	Verify DunningProcessorService runs on schedule	Service processes past-due subscriptions periodically	High
DUN-05	Dunning stops on successful payment	Payment succeeds during dunning process	Dunning halted, subscription remains active	High

10. Middleware Testing

10.1 Rate Limiting

TC ID	Test Case	Steps / Input	Expected Result	Priority
MW-01	Under rate limit	Send requests below RATE_LIMIT_REQUESTS_PER_MINUTE (60)	All requests succeed (200 OK)	Critical
MW-02	Exceed rate limit	Send 61+ requests in one minute to same endpoint	429 Too Many Requests after limit exceeded	Critical
MW-03	Rate limit reset after window	Exceed limit, wait 60 seconds, retry	Request succeeds after window reset	High
MW-04	Per-endpoint rate limit configuration	Verify different endpoints have different limits (from EndpointRateLimitConfig)	Limits applied per-endpoint configuration	Medium

10.2 Tenant Resolution Middleware

TC ID	Test Case	Steps / Input	Expected Result	Priority
MW-05	Valid X-Tenant-Id header	Send request with valid tenant ID	Request scoped to correct tenant	Critical
MW-06	Missing X-Tenant-Id on tenant-scoped endpoint	Omit X-Tenant-Id header on /api/v1/customers	400 Bad Request or resolved from JWT claims	High
MW-07	Non-existent tenant ID	Send X-Tenant-Id with UUID that does not exist	404 or 400, request rejected	High

10.3 Idempotency Middleware

TC ID	Test Case	Steps / Input	Expected Result	Priority
MW-08	First request with idempotency key	POST with X-Idempotency-Key: unique-uuid	Request processed normally, result cached	High
MW-09	Replay with same idempotency key	Same POST with same X-Idempotency-Key	Cached response returned, no side effects	High
MW-10	Different method same key	GET then POST with same idempotency key	Each method tracked independently	Medium

10.4 Request Logging Middleware

TC ID	Test Case	Steps / Input	Expected Result	Priority
MW-11	API call logged to database	Make any API request, check ApiCallLog table	Request logged with method, path,	High

			status, duration, tenant	
MW-12	Sensitive data not logged	POST login with password, check logs	Password not stored in log records	Critical
MW-13	Log entries viewable via API	GET /api/v1/logs as Admin	200 OK, API call logs returned with filtering	Medium

11. Background Services Testing

TC ID	Test Case	Steps / Input	Expected Result	Priority
BG-01	WebhookDispatcherService lifecycle	Start application, verify service starts and processes queue	Service running, dispatches outbound webhooks	Critical
BG-02	WebhookRetryService processes failed deliveries	Create failed webhook delivery, verify retry service picks it up	Failed webhooks retried with exponential backoff	Critical
BG-03	DunningProcessorService scheduled execution	Verify service runs on configured interval	Processes past_due subscriptions per dunning schedule	High
BG-04	Background service resilience	Simulate transient error in background service	Service recovers, continues processing	High
BG-05	Graceful shutdown	Send SIGTERM during webhook processing	Current operations complete before shutdown	Medium

12. Analytics and Dashboard Testing

12.1 Revenue Analytics

TC ID	Test Case	Steps / Input	Expected Result	Priority
AN-01	MRR (Monthly Recurring Revenue)	GET /api/v1/analytics/revenue/mrr	Correct sum of all active subscription monthly amounts	High
AN-02	ARR (Annual Recurring Revenue)	GET /api/v1/analytics/revenue/arr	MRR x 12 correctly calculated	High
AN-03	Churn rate calculation	GET /api/v1/analytics/revenue/churn	Percentage of canceled subscriptions over period	High
AN-04	Customer Lifetime Value	GET /api/v1/analytics/revenue/clv	Average revenue per customer calculated correctly	Medium
AN-05	Dashboard summary	GET /api/v1/dashboard/summary	Aggregated counts: customers, payments, revenue, subscriptions	High

12.2 SignalR Real-Time Updates

TC ID	Test Case	Steps / Input	Expected Result	Priority
RT-01	SignalR hub connection	Connect to /hubs/dashboard with valid JWT token	Connection established, join tenant group	High
RT-02	Real-time payment notification	Process payment, verify SignalR message sent	Connected clients receive payment update in real-time	High
RT-03	Tenant-scoped notifications	Process Tenant 1 payment, verify Tenant 2 does not receive	Notifications scoped to correct tenant group	Critical
RT-04	SignalR reconnection	Disconnect and reconnect client	Client resumes receiving notifications	Medium

12.3 Export

TC ID	Test Case	Steps / Input	Expected Result	Priority
EXP-01	Export payments to CSV	POST /api/v1/exports with type=payments, format=csv	CSV file with all tenant payment data	High
EXP-02	Export with date range filter	POST /api/v1/exports with startDate and endDate	Only data within range exported	Medium

EXP-03	Export large dataset performance	Export 1000+ records	Completes within acceptable time, no timeout	Medium
--------	----------------------------------	----------------------	--	--------

13. Stripe Connect and Tax Configuration Testing

13.1 Stripe Connect

TC ID	Test Case	Steps / Input	Expected Result	Priority
CON-01	List connected accounts	GET /api/v1/connect/accounts	200 OK, returns 3 seeded Connect accounts (express + standard)	High
CON-02	Create Connect account link	POST /api/v1/connect/accounts/{id}/link	Stripe onboarding URL returned	High
CON-03	Fee configuration per account	Verify platform fee settings per connected account	Application fee correctly applied on payments	Medium

13.2 Tax Configuration

TC ID	Test Case	Steps / Input	Expected Result	Priority
TAX-01	Tax configuration for tenant	GET /api/v1/tax/config	200 OK, Stripe Tax config with EU reverse-charge settings	High
TAX-02	Tax calculation on invoice	Create invoice for EU customer with tax exemption	Tax calculated correctly, exemption applied	High
TAX-03	Update tax configuration	PUT /api/v1/tax/config with new tax settings	200 OK, future invoices use updated config	Medium

14. Email Template and Notification Testing

TC ID	Test Case	Steps / Input	Expected Result	Priority
EM-01	List email templates (6 templates)	GET /api/v1/email/templates	Returns welcome, invoice, payment_failure, retry, cancellation, and other templates	High
EM-02	Update email template	PUT /api/v1/email/templates/{id} with new subject and body	Template updated, used in subsequent sends	High
EM-03	Payment failure email triggered	Trigger payment failure event	Payment failure email queued/sent to customer	High
EM-04	Welcome email on customer creation	Create new customer	Welcome email template used for notification	Medium
EM-05	Template variable substitution	Verify template renders with customer name, amount, date, etc.	All placeholders replaced with actual data	Medium

15. SuperAdmin Module Testing

TC ID	Test Case	Steps / Input	Expected Result	Priority
SA-01	SuperAdmin dashboard	Login as SuperAdmin, GET /api/v1/super-admin/dashboard	Cross-tenant aggregated metrics	High
SA-02	Create new tenant	POST /api/v1/super-admin/tenants with tenant details	201 Created, new tenant with admin user	Critical
SA-03	List all tenants	GET /api/v1/super-admin/tenants	All tenants listed with metadata	High
SA-04	Update tenant settings	PUT /api/v1/super-admin/tenants/{id}	Tenant configuration updated	Medium
SA-05	Global analytics	GET /api/v1/super-admin/analytics (global revenue, users, etc.)	Platform-wide aggregated analytics	High
SA-06	Global email templates	CRUD on global email templates via SuperAdmin	Templates managed globally, inherited by tenants	Medium
SA-07	Platform settings	GET/PUT /api/v1/super-admin/settings	Platform-level configuration managed	Medium
SA-08	Non-SuperAdmin access denied	Login as Admin, access /api/v1/super-admin/*	403 Forbidden on all SuperAdmin endpoints	Critical

16. Frontend UI Testing

16.1 Authentication UI

TC ID	Test Case	Steps / Input	Expected Result	Priority
UI-01	Login page renders	Navigate to /login	Email, password fields, and login button visible	Critical
UI-02	Successful login redirects to dashboard	Login with valid credentials	Redirect to / (dashboard), sidebar and header visible	Critical
UI-03	Invalid login shows error	Submit wrong credentials	Error message displayed, no redirect	High
UI-04	Protected routes redirect to login	Navigate to /payments without auth	Redirected to /login	Critical
UI-05	Role-based sidebar items	Login as each role, check sidebar links	Viewer: no Users/Settings links. Manager: no Users. Admin: all links.	High

16.2 Dashboard

TC ID	Test Case	Steps / Input	Expected Result	Priority
UI-06	Dashboard loads with metrics	Login, navigate to /	Revenue, customers, subscriptions, payments cards visible	High
UI-07	Charts render correctly	Check Chart.js charts on dashboard	Revenue chart, subscription chart render with data	High
UI-08	Real-time update via SignalR	Create payment in another session	Dashboard metrics update without page reload	Medium

16.3 Billing Pages

TC ID	Test Case	Steps / Input	Expected Result	Priority
UI-09	Payments list page	Navigate to /payments	Table with payment records, search, filters, pagination	High
UI-10	Subscriptions page	Navigate to /subscriptions	Active, trialing, past_due, canceled subscriptions displayed	High

UI-11	Customers page with search	Navigate to /customers, type in search box	Filtered customer list updates dynamically	High
UI-12	Invoices page	Navigate to /invoices	Invoice list with status badges and amounts	High
UI-13	Refunds page	Navigate to /refunds	Refund records with status: succeeded, pending, partial	Medium
UI-14	Plans page CRUD	Navigate to /plans, create/edit/view plans	All plan operations from UI working	High
UI-15	Coupons page	Navigate to /coupons	Coupon list with promotion codes	Medium
UI-16	Usage billing page	Navigate to /usage-billing	Usage records and metered data visible	Medium
UI-17	Credits page	Navigate to /credits	Customer credit balances and transaction history	Medium
UI-18	Dunning page	Navigate to /dunning	Dunning schedule steps and recovery status	Medium
UI-19	Export center	Navigate to /exports	Export options with format and date range selection	Medium
UI-20	Connect page (Admin only)	Navigate to /connect as Admin	Stripe Connect accounts listed	Medium

16.4 Gateway and Admin Pages

TC ID	Test Case	Steps / Input	Expected Result	Priority
UI-21	API Keys page	Navigate to /api-keys as Admin	API keys listed with masked values, create button	High
UI-22	Logs page	Navigate to /logs	API call logs with filtering and pagination	Medium
UI-23	Webhooks page	Navigate to /webhooks	Webhook subscriptions with delivery status	High
UI-24	Webhook events page	Navigate to /webhook-events	Inbound/outbound event log	Medium
UI-25	Users management page	Navigate to /users as Admin	User list with role badges, CRUD operations	High
UI-26	Settings page	Navigate to /settings as Admin	Tenant settings form, save functionality	High

UI-27	Audit log page	Navigate to /audit-logs as Admin	Audit trail with user, action, timestamp	Medium
UI-28	Tax config page	Navigate to /tax-config as Admin	Tax configuration form with Stripe Tax settings	Medium
UI-29	Email templates page	Navigate to /email-templates as Admin	Template editor with preview	Medium
UI-30	Analytics page	Navigate to /analytics	Revenue analytics with MRR, ARR, churn charts	High

16.5 SuperAdmin Pages

TC ID	Test Case	Steps / Input	Expected Result	Priority
UI-31	SuperAdmin dashboard	Login as SuperAdmin, navigate to /super-admin/dashboard	Platform-wide overview metrics	High
UI-32	Tenants management page	Navigate to /super-admin/tenants	All tenants listed, create new tenant option	High
UI-33	Global analytics page	Navigate to /super-admin/analytics	Cross-tenant analytics and charts	Medium
UI-34	Platform settings page	Navigate to /super-admin/settings	Global platform configuration	Medium

17. Input Validation Testing

The application uses FluentValidation with 16+ validator classes. Each validator must be tested for both valid inputs and boundary conditions.

TC ID	Test Case	Steps / Input	Expected Result	Priority
VAL-01	CreateCustomerValidator - missing name	POST /api/v1/customers with empty name field	400 Bad Request: Name is required	High
VAL-02	CreateCustomerValidator - invalid email	POST /api/v1/customers with email='notanemail'	400 Bad Request: Invalid email format	High
VAL-03	CreatePaymentIntentValidator - zero amount	POST /api/v1/payments/create-intent with amount=0	400 Bad Request: Amount must be positive	High
VAL-04	CreatePaymentIntentValidator - negative amount	POST /api/v1/payments/create-intent with amount=-100	400 Bad Request	High
VAL-05	CreateSubscriptionValidator - missing planId	POST /api/v1/subscriptions without planId	400 Bad Request: PlanId required	High
VAL-06	CreatePlanValidator - invalid interval	POST /api/v1/plans with unsupported interval	400 Bad Request: Invalid interval	Medium
VAL-07	LoginRequestValidator - empty password	POST /api/v1/auth/login with empty password	400 Bad Request: Password required	High
VAL-08	CreateWebhookSubscriptionValidator - invalid URL	POST /api/v1/webhooks with url='not-a-url'	400 Bad Request: Invalid URL format	Medium
VAL-09	CreateRefundValidator - missing paymentId	POST /api/v1/refunds without paymentId	400 Bad Request: PaymentId required	Medium
VAL-10	CreateCouponValidator - percent_off > 100	POST /api/v1/coupons with percent_off=150	400 Bad Request: Percent off must be 1-100	Medium
VAL-11	SQL injection in search parameters	GET /api/v1/customers?search='; DROP TABLE Customers;--	Parameterized query prevents injection	Critical
VAL-12	XSS in customer name	POST /api/v1/customers with name='<script>alert(1)</script>'	Input sanitized or encoded, no script execution	Critical

18. Security Testing

TC ID	Test Case	Steps / Input	Expected Result	Priority
SEC-01	SQL injection on all input fields	Test all text inputs with SQL payloads via fuzzing	No SQL injection possible (EF Core parameterized)	Critical
SEC-02	Cross-Site Scripting (XSS)	Submit XSS payloads in customer names, plan descriptions, etc.	All output encoded, no script execution in frontend	Critical
SEC-03	CSRF protection	Attempt state-changing requests from different origin without proper CORS	Requests blocked by CORS policy	High
SEC-04	CORS configuration validation	Verify CORS_ALLOWED_ORIGINS enforced (localhost:3000 only in dev)	Cross-origin requests from unauthorized origins rejected	High
SEC-05	JWT secret key strength	Verify JWT_SECRET_KEY is >= 32 characters, not default	Strong key in production, not Demo@123 etc.	Critical
SEC-06	Password storage security	Check user table for plaintext passwords	Passwords stored as bcrypt/PBKDF2 hashes only	Critical
SEC-07	Sensitive data in logs	Review Seq logs for passwords, tokens, card numbers	No sensitive data in structured logs	Critical
SEC-08	API rate limiting prevents brute force	Attempt 100+ login requests rapidly	Rate limited after threshold, 429 returned	High
SEC-09	Stripe secret key not exposed	Check frontend bundle, API responses, logs for sk_test_*	Secret key never exposed to client	Critical
SEC-10	HTTPS enforcement in production	Access API via HTTP in production config	Redirected to HTTPS or rejected	High
SEC-11	HTTP security headers	Check response headers (X-Content-Type-Options, X-Frame-Options, etc.)	Security headers present and correctly configured	High
SEC-12	Encryption service for sensitive data	Verify EncryptionService used for API keys and secrets at rest	Sensitive data encrypted in database	High

19. Performance and Load Testing

TC ID	Test Case	Steps / Input	Expected Result	Priority
PERF-01	API response time (P95 < 500ms)	Benchmark all major endpoints under normal load	95th percentile response < 500ms	High
PERF-02	Concurrent user load test (100 users)	Simulate 100 concurrent users performing CRUD operations	No errors, response time degrades < 3x	High
PERF-03	Database query performance	Enable EF Core query logging, identify N+1 queries	No N+1 queries, proper eager loading used	High
PERF-04	Webhook dispatch throughput	Queue 1000 outbound webhooks, measure dispatch rate	All dispatched within acceptable window	Medium
PERF-05	SignalR connection scaling	Connect 200+ SignalR clients simultaneously	All connections maintained, notifications delivered	Medium
PERF-06	Large dataset pagination	Query /api/v1/payments with 10,000+ records	Pagination works, no memory issues	High
PERF-07	Frontend initial load time	Measure time to interactive (TTI) with Lighthouse	TTI < 3 seconds on standard connection	Medium
PERF-08	Frontend lazy loading	Verify code splitting on route navigation	Each page chunk loads on-demand, not upfront	Medium
PERF-09	Memory leak detection	Run API under sustained load for 1 hour	No steadily increasing memory usage	Medium

20. Integration and End-to-End Testing

20.1 Critical End-to-End Flows

TC ID	Test Case	Steps / Input	Expected Result	Priority
E2E-01	Full payment lifecycle	Create customer -> Create checkout -> Complete payment -> Verify invoice -> Issue refund	Complete flow succeeds, all records consistent	Critical
E2E-02	Full subscription lifecycle	Create plan -> Subscribe customer -> Renewal (webhook) -> Cancel -> Verify final invoice	Subscription goes through all states correctly	Critical
E2E-03	Failed payment dunning recovery	Create subscription -> Payment fails -> Dunning emails -> Retry succeeds	Subscription recovered via dunning process	High
E2E-04	New tenant onboarding	SuperAdmin creates tenant -> Creates admin user -> Admin logs in -> Configures Stripe -> Creates first customer and payment	Complete onboarding flow succeeds	High
E2E-05	Multi-tenant concurrent operations	Both tenants perform operations simultaneously	No cross-tenant data leakage, all operations succeed	Critical
E2E-06	Coupon applied to subscription	Create coupon -> Create promotion code -> Subscribe with code -> Verify discounted invoice	Discount applied correctly on first/all invoices per config	High
E2E-07	Usage billing end-to-end	Subscribe to metered plan -> Record usage -> Billing cycle -> Invoice with usage	Invoice reflects recorded usage quantity x rate	High

20.2 Stripe Integration Tests

TC ID	Test Case	Steps / Input	Expected Result	Priority
INT-01	Stripe API connectivity	Verify connection to Stripe with test keys	Successful API calls, no auth errors	Critical
INT-02	Stripe CLI webhook forwarding	stripe listen --forward-to localhost:58492/api/v1/webhooks/stripe	Events forwarded and processed locally	Critical
INT-03	Stripe test card numbers	Test with 4242424242424242 (success), 4000000000000002 (decline)	Success card succeeds, decline card fails correctly	Critical
INT-04	Stripe sync consistency	Create customer via API, verify exists on Stripe dashboard	Local and Stripe records match	High
INT-05	Stripe error handling	Send invalid request to Stripe (e.g., negative amount)	Stripe error mapped to appropriate API error response	High

21. Docker and Deployment Testing

TC ID	Test Case	Steps / Input	Expected Result	Priority
DOC-01	Docker Compose full stack startup	docker-compose up -d from project root	SQL Server, API, Frontend, Seq all start within 60s	Critical
DOC-02	Database migration on startup	Start API with empty database	EF Core migrations run automatically, tables created	Critical
DOC-03	Seed data loaded on fresh start	Verify seeded data after fresh docker-compose up	2 tenants, 6 users, 15 customers, 75+ payments, etc.	High
DOC-04	Health check endpoint	GET /api/v1/health	200 OK with service health status	High
DOC-05	Environment variable configuration	Set all vars from .env.example, verify app uses them	Stripe keys, JWT secret, DB connection all configurable via env	High
DOC-06	Seq logging accessible	Navigate to http://localhost:5341	Structured logs visible and searchable	Medium
DOC-07	Container restart recovery	docker-compose restart api	API recovers, reconnects to DB and Stripe	Medium

22. Existing Unit Tests and Coverage Gaps

22.1 Current Test Coverage

The project has an existing test suite using xUnit, Moq, and FluentAssertions. Current coverage includes:

- CreditServiceTests (5 tests): Add credit, adjust balance, tenant/user assignment, refund-to-credit stub
- DunningServiceTests: Dunning schedule and processing logic
- EmailServiceTests: Email template rendering and sending
- UsageBillingServiceTests: Usage record creation and billing calculations
- IdempotencyMiddlewareTests: Idempotency key caching and replay behavior

22.2 Critical Coverage Gaps

The following service and controller layers currently have NO unit test coverage and should be prioritized:

- AuthService: Login, registration, token refresh, password change
- StripePaymentGateway: Checkout sessions, payment intents, Stripe API error handling
- SubscriptionService: Create, cancel, update, lifecycle management
- CustomerService: CRUD operations, Stripe sync, search
- InvoiceService: Auto-generation, status management, line items
- RefundService: Full/partial refund logic, amount validation
- WebhookSubscriptionService and WebhookDispatchService: Outbound webhook CRUD and dispatch
- StripeWebhookHandler: Inbound event parsing, routing, and processing
- RevenueAnalyticsService: MRR, ARR, churn, CLV calculations
- TenantMiddleware: Tenant resolution and isolation enforcement
- RateLimitMiddleware: Rate limit enforcement and configuration
- ApiKeyAuthMiddleware: API key validation and tenant mapping
- All Controllers: HTTP status code mapping, authorization attribute enforcement

22.3 Recommended Test Coverage Targets

Layer	Current	Target	Priority	Est. Tests
Services (Business Logic)	~15%	80%+	Critical	200+
Controllers (API Layer)	0%	70%+	High	100+
Middleware	~10%	90%+	Critical	40+
Validators	0%	100%	High	60+
Background Services	0%	70%+	High	30+
Integration Tests (API)	0%	60%+	High	80+
Frontend Components	0%	50%+	Medium	50+

23. Test Execution Plan

23.1 Execution Phases

Phase	Scope	Activities	Duration	Dependencies
Phase 1	Unit Tests for Services	Write tests for all 30+ services using Moq + FluentAssertions	2 weeks	None
Phase 2	Middleware + Validator Tests	Test all 6 middleware components and 16+ validators	1 week	Phase 1
Phase 3	Integration Tests (API)	WebApplicationFactory tests for all controller endpoints	2 weeks	Phase 1-2
Phase 4	Stripe Integration	End-to-end tests with Stripe test mode and CLI	1 week	Stripe test keys
Phase 5	Frontend Testing	Component tests with React Testing Library + Vitest	2 weeks	Phase 3
Phase 6	Security + Performance	Penetration testing, load testing with k6/JMeter	1 week	All phases

23.2 Tools and Frameworks

- Backend Unit/Integration: xUnit, Moq, FluentAssertions, EF InMemory, WebApplicationFactory
- Frontend: Vitest, React Testing Library, MSW (Mock Service Worker)
- E2E: Playwright or Cypress for full browser automation
- Performance: k6, Apache JMeter, or NBomber for .NET
- Security: OWASP ZAP, Burp Suite for API scanning
- Stripe: Stripe CLI (stripe trigger, stripe listen) for webhook simulation
- CI/CD: GitHub Actions or Azure DevOps pipeline integration

23.3 Test Data Strategy

The application seeds comprehensive demo data on startup, which serves as the baseline for all tests. For unit tests, mock repositories using Moq. For integration tests, use EF Core InMemory provider or a dedicated SQL Server test database. For Stripe integration tests, always use test mode API keys (pk_test_ / sk_test_) and Stripe CLI for webhook simulation.

23.4 Defect Severity Classification

Severity	Definition	Example	SLA
S1 - Blocker	System unusable, data loss, security breach	Cross-tenant data leak, payment double-charge	Fix within 4 hours

S2 - Critical	Major feature broken, no workaround	Cannot create subscriptions, webhooks not processing	Fix within 24 hours
S3 - Major	Feature impaired, workaround exists	Filter not working on payments page, export timeout	Fix within 1 week
S4 - Minor	Cosmetic or low-impact issue	UI alignment, spelling errors, minor UX issues	Next release

This concludes the complete testing document for the Stripe Billing Service. The total test case count across all sections is approximately 200+ test cases covering unit, integration, security, performance, and end-to-end scenarios. Executing all phases will require approximately 9 weeks of dedicated testing effort.